Software Requirements Specification (SRS)

Student Project Portfolio & Showcase App | Problem Statement #04 | Version 1.0

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of the Student Project Portfolio & Showcase App, a web/mobile system used at a student project expo. It is intended for the project team, evaluators and anyone who will design, build or test the system.

1.2 Scope

The system lets student teams publish showcase profiles, lets panel judges score projects with a digital rubric, lets the public vote once per project after OTP identity verification, and shows a live leaderboard. It also flags suspicious voting or scoring patterns for organiser review. Payment, project hosting and user-to-user messaging are out of scope.

1.3 Definitions and Abbreviations

Term	Meaning
FR / NFR	Functional / Non-Functional Requirement
OTP	One-Time Password sent to a mobile number or email
Anti-sybil	Protection against one person or script creating many fake identities or votes
Rubric	Weighted set of judging criteria (innovation, technical execution, presentation)
RTM	Requirements Traceability Matrix
UC	Use Case

1.4 References

- Requirements Table (FR-001 – FR-005, NFR-001 – NFR-002)
- UML Use-Case Diagram (UC-01 – UC-07)
- Use-Case Flow Specification – UC-05 Cast Public Vote
- UML Component Diagram and Architecture Justification (folder 2)
- Requirements Traceability Matrix (folder 1)

2. Overall Description

2.1 Product Perspective

A new, self-contained web and mobile application. It depends on two external services: the GitHub API (to verify repository URLs) and an SMS/Email gateway (to deliver OTPs). The chosen architecture is microservices behind an API Gateway.

2.2 User Classes (Actors)

Actor	Description	Use Cases
Student Team	Creates and maintains the project's showcase profile	UC-01, UC-02, UC-07
Panel Judge	Scores projects against the digital rubric	UC-03, UC-07
Public Voter	Browses projects, verifies identity by OTP and votes	UC-05, UC-06, UC-07
Organiser (system-	Reviews projects flagged automatically by the system	UC-04

triggered)		

2.3 Operating Environment

Modern web browsers and mobile devices with an internet connection. Server side runs as independently deployable services.

2.4 Assumptions and Dependencies

- Project profiles are published before the public voting window opens.
- Voters have a mobile number or email able to receive an OTP.
- The GitHub API and SMS/Email gateway are available during the expo.

3. Use Cases

ID	Use Case	Actor	Relationship
UC-01	Create Showcase Profile	Student Team	«include» UC-02
UC-02	Validate Repository URL	(included)	Included by UC-01
UC-03	Grade Project via Digital Rubric	Panel Judge	Extended by UC-04
UC-04	Flag Project for Manual Review	System / Organiser	«extend» UC-03
UC-05	Cast Public Vote	Public Voter	«include» UC-06
UC-06	Verify Voter Identity (OTP)	(included)	Included by UC-05
UC-07	View Real-Time Leaderboard	Team, Judge, Voter	—

UC-05 summary: Voter selects a project and taps Vote; the system checks for a prior vote, requests a contact detail, sends an OTP, validates it (including expiry), records the vote, updates the count and triggers leaderboard recalculation. Alternate flow 7a: an invalid or expired OTP shows an error and allows retry or a new code; after three failures the attempt is cancelled and the voter returns to the catalog. Full detail is in the UC-05 flow document.

4. Specific Requirements

4.1 Functional Requirements

ID	Requirement	Priority	Acceptance Criteria
FR-001	The system shall enable student teams to create showcase profiles containing abstract summaries, demo video links and verified GitHub repository URLs.	High	Pass: profile is published once the repository link is validated. Fail: an invalid URL format is accepted.
FR-002	The system shall allow a Panel Judge to score a submitted project against a weighted digital rubric (innovation, technical execution, presentation).	High	Pass: scores for every criterion are submitted and the total is saved. Fail: a score is accepted with a criterion blank.
FR-003	The system shall allow a member of the public to cast one vote per project only after identity verification by a one-time OTP sent to a registered mobile number or email.	High	Pass: verified vote is recorded and a second attempt by the same identity is rejected. Fail: the same identity can vote twice.
FR-004	The system shall recalculate each project's leaderboard rank by combining normalised judge scores and public vote counts using a fixed weighting formula.	Medium	Pass: rank updates automatically within the NFR-001 latency window after a new score or vote. Fail: rank is unchanged.
FR-005	The system shall automatically place a project in an organiser review queue if its votes grow past a threshold within a short window, or if the spread of judge scores exceeds a limit.	Medium	Pass: a qualifying project appears in the queue within one minute. Fail: it never appears.

4.2 Non-Functional Requirements

ID	Requirement	Priority	Acceptance Criteria
NFR-001	Public voting and leaderboard calculations shall update in real time with an API response latency under 300 ms.	High	Pass: benchmarking confirms the latency target under simulated peak load.
NFR-002	The system shall remain available for at least 3,000 concurrent users during peak hours with a failed-request rate no higher than 0.5%.	High	Pass: load test with 3,000 simulated users finishes at or below 0.5% failures.

4.3 External Interfaces

- User interface: responsive web / mobile screens for catalog, voting, judging and leaderboard.
- GitHub REST API (HTTPS) for repository URL verification.
- SMS / Email gateway API (HTTPS) for OTP delivery.
- Internal service APIs: REST/JSON through the API Gateway; events over AMQP via the Event Bus.

5. System Architecture Overview

The system uses a microservices architecture: Profile, OTP Verification, Voting, Judging, Leaderboard and Review Monitor services behind an API Gateway, communicating asynchronously through an Event Bus. See Component_Diagram.png and Architecture_Justification.pdf in folder 2.

6. Traceability

Every requirement is traced to use cases, components and planned test cases in RTM_Table.docx (folder 1).